
Oliver Hauke

ist Mathematiker und IT-Referent im Kompetenzzentrum „Analyse und Auswertung“ des Statistischen Bundesamtes. Er verantwortet die Weiterentwicklung der technischen Infrastruktur des Forschungsdatenzentrums und berät das Netzwerk empirische Steuerforschung in der effizienten Nutzung der Systeme.

Annette Kristiansen

ist Ökonomin und Referentin im Referat „Unternehmenssteuern“ des Statistischen Bundesamtes. Sie beschäftigt sich mit der Zusammenführung der Unternehmenssteuerstatistiken und betreut das Netzwerk empirische Steuerforschung. Für ihre externe Promotion an der Universität Bremen untersucht sie die technologische Vielfalt multinationaler Unternehmen.

EFFIZIENTE AUSWERTUNG DES BUSINESS-TAX-PANELS MITHILFE VON R

Oliver Hauke, Annette Kristiansen

✦ **Schlüsselwörter:** Effiziente Analysen, R-Programmierung, Forschungsdaten, Unternehmenssteuerstatistik, Business-Tax-Panel, Netzwerk empirische Steuerforschung

ZUSAMMENFASSUNG

Das Forschungsdatenzentrum des Statistischen Bundesamtes baut seine technische Infrastruktur gezielt aus, um der Wissenschaft erweiterte Analysemöglichkeiten amtlicher Daten bereitzustellen. Seit November 2025 steht Forschenden exklusiver Zugang zu diesen erweiterten Systemressourcen in R und Stata zur Verfügung. R-basierte Tools – insbesondere Apache Arrow, Parquet und DuckDB – ermöglichen es, gängige Analysen großer Forschungsdatensätze in Echtzeit auszuführen. Am Beispiel des Business-Tax-Panel (2013 bis 2019) werden die erzielbaren Effizienzgewinne bei der Auswertung umfangreicher steuerstatistischer Daten im Rahmen des Netzwerks empirische Steuerforschung demonstriert.

✦ **Keywords:** *Efficient analysis – R programming – research data – corporate tax statistics – Business Tax Panel – empirical tax research network*

ABSTRACT

The Research Data Centre at the Federal Statistical Office is strategically expanding its technical infrastructure to provide researchers with enhanced analytical capabilities for official data. Since November 2025, researchers have had exclusive access to these extended system resources in R and Stata. R-based tools – particularly Apache Arrow, Parquet, and DuckDB – enable real-time analysis of large research datasets. Using the Business Tax Panel (2013–2019), this paper demonstrates the achievable efficiency gains in analysing large-scale tax statistics within the empirical tax research network.

Inhaltsverzeichnis

1 Performanzvergleich	2
1.1 Versuchsaufbau	2
1.2 Vergleichswert in SAS und Stata	2
1.3 Kleiner Vergleich zwischen R, SAS und Stata	3
1.4 Größere Datenmengen in R	5
1.5 (Weitere Erkenntnisse des Experiments)	6

1

Performanzvergleich

1.1 Versuchsaufbau

Um zwischen den in Abschnitt 2.1 vorgestellten Datenverarbeitungsmethoden abzuwägen, haben wir deren Performanz in einem systematischen Vergleich der Performance auf dem SAS-Server des Statistischen Bundesamtes und den neuen OnSite R- und Stata-Servern des FDZ gemessen. Weiterführende Performanzmessung sowie der vollständig reproduzierbare Code sind auf [Opencode](https://www.opencode.de/URL-To-Be-Announced) verfügbar¹.

Für die Untersuchung wurden simulierte Einzeldaten in verschiedenen Größen (0,1%/1%/10% von den insgesamt 80 Mio. Beobachtungen des BTP) betrachtet. Sie wurden aus öffentlich verfügbaren Informationen generiert, um die wesentlichen Strukturen des BTP für unsere Analyse künstlich nachzubilden. Abgrenzend zu den im FDZ bereitgestellten *Datenstrukturfiles* wurden die Testdaten nicht aus echten Mikrodaten abgeleitet. Der simulierte Datensatz ist ausschließlich für technische Testzwecke der nachfolgenden Anwendungsfälle bestimmt:

1. **Fallzahlen** (pro Statistik und Jahr),
2. **Regression** (Einflussfaktoren auf Verlustvorträge).

Um eine zuverlässige Performanz-Messung zu erhalten, wurde jede Datenverarbeitungsmethode dreifach angewandt. Der Vergleich erfolgt anhand der mittleren Messung der folgenden Zielgrößen:

1. **Arbeitsspeicher** der Auswertung,
2. **Laufzeit** der Auswertung (real verstrichene Zeit),
3. **Festplattenspeicher** der verwendeten Daten.

Die Zielgrößen sind in dieser Reihenfolge zu priorisieren, da ein übermäßiger Arbeitsspeicherverbrauch zu schwerwiegenden Abbrüchen führen kann, während hohe Laufzeiten dennoch zu einem erfolgreichen Ergebnis führen können. Festplattenspeicher ist ausreichend vorhanden, aber unnötige Belegung sollte vermieden werden.

Feine Abstufungen der Zielgrößen sind für unsere Aufgaben nicht gleichermaßen relevant. Eine Methode, die die betrachteten Auswertungen in unter 10 Sekunden mit weniger als 4 GB Arbeitsspeicher durchführen kann, wird bereits als optimal bewertet. Weitere Optimierungen sind in diesem Fall nicht erforderlich. Falls bereits die gewünschte Performanz erreicht wurde, rückt die Reduktion des Entwicklungsaufwand in den Vordergrund. Entsprechend sind Aspekte wie einfache Anwendbarkeit und Nachvollziehbarkeit – selbst für unerfahrene Nutzende – ausschlaggebend.

1.2 Vergleichswert in SAS und Stata

Klassisch werden die betrachteten Auswertungsaufgaben im Statistischen Bundesamt hauptsächlich mit SAS und im FDZ mit Stata gelöst. Um die nachfolgenden Ergebnisse einzuordnen, haben wir die Performanzmessungen für 1% des simulierten BTP-Daten (etwa. 800.000 Beobachtungen) in SAS und Stata durchgeführt.

¹www.opencode.de/URL-To-Be-Announced.

Tabelle 1

Baselines in SAS und Stata (anhand 1% des simuliertes BTP)

Software	Anwendung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
SAS	Fallzahl	65 MB	2.4 Min.	33 GB
	Regression	7 MB	2.8 Min.	33 GB
Stata	Fallzahl	33 GB	55,2 Sek.	32 GB
	Regression	33 GB	55,8 Sek.	32 GB

Tabelle 2

Performanz Messungen für die Fallzahlberechnung anhand 1% des BTP

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{polars}	Parquet	-	3,3 MB	1,0 Sek	4,9 GB
{duckdb}	Parquet	-	4,8 MB	1,5 Sek	4,9 GB
{arrow}	Parquet	Variablenauswahl	273,7 MB	1,7 Sek	4,9 GB
{data.table}	CSV	Variablenauswahl	42,9 MB	2,5 Sek	13,0 GB
{arrow}	Parquet	Datenaufteilung (gleichmäßig)	229,7 MB	2,8 Sek	4,9 GB
{arrow}	Parquet	-	272,8 MB	2,8 Sek	4,9 GB
{arrow}	Parquet	-	230,4 MB	3,8 Sek	4,9 GB
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	229,9 MB	3,9 Sek	4,8 GB
{data.table}	CSV	-	33,7 GB	7,4 Sek	13,0 GB
{data.table}	CSV	Datenaufteilung (Berichtsjahr)	50,4 GB	13,6 Sek	10,5 GB

Die Ergebnisse in Tabelle 1 zeigen, dass der umfangreiche Arbeitsspeicher des FDZ OnSite-Servers nicht ausreicht, um das ganze BTP in Stata einzulesen, sodass zwingend mit Variablenselektion gearbeitet werden muss. SAS zeigt seine Stärke anhand der geringen Beanspruchung des Arbeitsspeichers, benötigt aber mit etwa 2,3 Minuten pro Auswertung die doppelte Laufzeit gegenüber Stata.

1.3 Kleiner Vergleich zwischen R, SAS und Stata

Für 1% der simulierten BTP Daten, reduzieren die R-Datenverarbeitungsmethode alle Zielgrößen gegenüber SAS und Stata erheblich (siehe Tabelle 2 und Tabelle 3). Selbst klassische R-Methoden sind um den Faktor 7 schneller und benötigen nur 33 % des Festplattenspeichers. {polars}, {duckdb} und {arrow} lösen die betrachteten Auswertungen sogar

- › in **unter 2 Sekunden** (27-fach schneller),
- › mit **unter 300 MB Arbeitsspeicher** – {duckdb} und {polars} sogar mit **unter 5 MB** (10-fach weniger)¹² –
- › und nur mit **5 GB Festplattenspeicher** (6-fach weniger gegenüber SAS und Stata).

Unter den klassischen R-Methoden bietet {data.table} die beste Performanz. Dabei ist wesentlich zu beachten, dass die Einlesefunktion `fread` direkt die relevanten Variablen auswählt. Dadurch lassen sich unsere Auswertungen um einen Faktor 3 beschleunigen und vor allem der Arbeitsspeicher um einen Faktor 1.000 reduzieren.

Alle hoch-performanten Methoden sind dermaßen effizient, dass die Datenmenge von 800.000 Beobachtungen nicht mehr ausreicht, um zwischen den besten Methoden zu differenzieren. Daher erhöhen wir im nächsten Experiment die Datenmenge auf 8.000.000 Beobachtungen (10% BTP).

¹² Lediglich in der Arbeitsspeicherbeanspruchung, ist SAS deutlich besser als klassische R-Methoden. Gegenüber hoch-performanten R-Methoden benötigt SAS jedoch den 10-fachen Arbeitsspeicher.

¹³ {arrow} benötigt mit 274 MB deutlich mehr Arbeitsspeicher als {polars} und {duckdb}, aber kommt dennoch mit minimalen Entwicklungsumgebungen aus, bspw. einem standardmäßigen Laptop.

Tabelle 3

Performanz Messungen für die Regressionsberechnung anhand 1% des BTP

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	529,7 MB	3,8 Sek	4,8 GB
{dplyr}	CSV	Variablenauswahl	154,8 MB	13,2 Sek	13,0 GB

1.4 Größere Datenmengen in R

Auch für 10 % der simulierten BTP-Daten, kann R die beiden betrachteten Auswertungen in Echtzeit mit minimalem Arbeitsspeicher beantworten (siehe Tabelle 4 und Tabelle 5). Die Auswertungsmethoden mit der höchsten Performanz nutzen alle das {parquet} und erreichen:

1. {arrow} benötigt nur **4,7 Sekunden** und 230 MB Arbeitsspeicher,
2. {duckdb} benötigt nur 5,1 Sekunden und **4,7 MB Arbeitsspeicher**.

Unerwarteterweise ist die Laufzeit von {polars} für diese Datenmenge mit 1,2 Minuten weit abgeschlagen, obwohl das gleiche Auswertungsskript für kleinere Datenmengen immer die geringste Laufzeit aufwies. Da {polars} das jüngste der betrachteten R-Packages ist, vermuten wir Performanzprobleme in der vorliegenden Version 1.0.1¹⁴. Zudem hat es noch größere Entwicklungsaufwände beansprucht, da der an Python angelehnte Syntaxstil deutlich von üblicher Programmierung in R abweicht. Öffentliche Hilfestellung bezieht sich hauptsächlich auf Python (oder Rust), was sich in den Antworten der KI-Chatassistenten widerspiegelt.

Klassische Methoden kommen bei der betrachteten Datenmenge bereits an ihre Grenzen. Falls in {data.table} unbedacht der vollständige Datensatz eingelesen wird, werden in unserem Anwendungsfall die verfügbaren 512 GB Arbeitsspeicher vollständig beansprucht. Wir klassische Methoden ist somit deutlich mehr Erfahrung der Nutzenden notwendig, um die relevanten Variablen manuell geschickt zu selektieren unter Beobachtung des Arbeitsspeicherverbrauchs.

Insgesamt kommen wir zu dem Ergebnis, dass {arrow} kombiniert mit {parquet} im Gesamtpaket die beste Datenverarbeitungsmethode für die Bewältigung unserer Aufgaben aus den folgenden Gründen ist:

1. Für die relevanteste Datenmenge zeigte {arrow} die geringsten Laufzeiten unter 5 Sekunden, was der Fachstatistik Antworten auf Sonderanfragen in Echtzeit ermöglicht und der Wissenschaft alle denkbaren explorativen Analysen.
 2. auch die Nutzererfahrung war mit {arrow} am besten und die damit einhergehenden Entwicklungsaufwände waren dadurch am geringsten.
 3. die anschauliche {tidyverse} Syntax lässt sich zum Großteil änderungsfrei und ohne Performanzverluste auf {arrow} übertragen. {duckdb} kann bei Kenntnissen in SQL zu bevorzugen sein und {polars} mit Erfahrungen in Python.
 4. Beide Anwendungen ließen sich in unter 20 nachvollziehbaren Zeilen in {arrow} programmieren.
 5. Obwohl nicht alle Features aus {tidyverse} für {arrow} verfügbar sind, haben wir bisher immer eine Lösung gefunden, um die gewünschte Auswertung umzusetzen, bspw. durch Kombination mit {duckdb}.
 6. {arrow} spart ausreichend Arbeitsspeicher (mit Verbrauch unter 1 GB) (tba: Fußnote zu duckdb/polars). Der Arbeitsspeicherverbrauch von {arrow} ist in unserer Auswertung nahezu konstant abhängig von der Datenmenge, sodass {arrow} leicht auf noch größere Datenmengen skalieren kann
 7. Dadurch das {arrow} die Nutzung auch von mehreren {parquet} Dateien unterstützt spart gegenüber CSV mind. 50% der Festplatte (für das reale BTP sogar 80%+) und ermöglicht schnelle Nutzung auch von Datenmengen überhalb des verfügbaren Arbeitsspeichers.
- › {arrow} zeigte im Experiment die beste Nutzendenerfahrung.
- › Beide Anwendungen lassen sich in unter 20 Z. mit Arrow programmieren (verständlich lesbar) s. Opencode Repo.
 - › Leicht und verständlich - adaptiert nativ schöne {tidyverse} Syntax. Dagegen sind {duckdb} und {polars} auf Zusatzpakete angewiesen, die tlw. die Performanz reduzieren können.
 - › Flexibel, selektives Einlesen ist zwingend erforderlich
 - › hohe Stabilität, fehlende Features können durch leicht in duckdb oder in den Arbeitsspeicher überführt und dennoch ausgeführt werden
- › kleiner Nachteile: Erfahrung notwendig, da...
- › erst Datenaufteilung/selektives Einlesen die maximale Performanz erreicht

¹⁴Das R-Package {polars} Version 1.0.1 war nur der erste Minor Release nach einer kompletten Überarbeitung. Inzwischen sind 5 neuere Major Releases mit diversen Performanz-Verbesserungen verfügbar.

Tabelle 4

Performanz Messungen für die Fallzahlberechnung anhand 10% des BTP

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{arrow}	Parquet	Datenaufteilung (gleichmäßig)	229,9 MB	4,7 Sek	48,8 GB
{duckdb}	Parquet	-	4,7 MB	5,1 Sek	49,4 GB
{arrow}	Parquet	Variablenauswahl	702,1 MB	7,5 Sek	49,4 GB
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	4,7 MB	11,7 Sek	48,1 GB
{arrow}	Parquet	-	230,4 MB	17,0 Sek	49,4 GB
{arrow}	Parquet	-	701,2 MB	20,6 Sek	49,4 GB
{data.table}	CSV	Variablenauswahl	427,5 MB	23,6 Sek	130,2 GB
{polars}	Parquet	-	3,3 MB	1,2 Min	49,4 GB
{data.table}	CSV	-	336,0 GB	2,2 Min	130,2 GB
{data.table}	CSV	Datenaufteilung (Berichtsjahr)	506,4 GB	5,0 Min	104,6 GB

Tabelle 5

Performanz Messungen für die Regressionsberechnung anhand 10% des BTP

Package	Dateiformat	Optimierung	Arbeitsspeicher	Laufzeit	Festplattenspeicher
{arrow}	Parquet	Datenaufteilung (Berichtsjahr)	782,1 MB	5,2 Sek	48,1 GB
{dplyr}	CSV	Variablenauswahl	1,5 GB	1,9 Min	130,2 GB

- › aktuell nicht der volle Funktionsumfang von {tidyverse} in {arrow} abgebildet wird, sodass auf fehlende Features mit {duckdb} etc. reagiert werden muss.

1.5 (Weitere Erkenntnisse des Experiments)

- › Auswahl der optimalen hoch-performanten Methode erfordert eine Untersuchung in der eigenen Infrastruktur anhand eigener Daten. Jedoch bieten alle HP-Methoden für die meisten Anwendungsfälle starke Performanzgewinne und sollten eingesetzt werden (nach den vorhandenen Erfahrungen und Strukturen: tidy vs Python vs SQL). Falls die Daten in CSV unter 25% des Arbeitsspeichers sind, sind auch klassische Methoden sehr gut nutzbar (bevorzugt {data.table}).
- › **Package für optimale Laufzeit ist nicht eindeutig.** Je nach Datenmenge hat sich die Top 3 stark unterschieden.
 - › 1e6: arrow > duckdb > DT+SEL > polars
 - › 1e5: polars > duckdb > arrow > DT+SEL
 - › 1e4: polars > data.table > arrow > readr
- › Unerwarteterweise ist {polars} bis 1% BTP am schnellsten aber für das 10% BTP abgeschlagen (unter den schlechtesten Methoden).
- › Dagegen ist **das effizienteste betrachtete Datenformat über 1000 Beobachtungen eindeutig Parquet** – sowohl in Speicherbedarf als auch in Lese- und Schreibzeiten.
- › HP-Methoden und Optimierungen können erst ab einer Mindestdatenmenge vorteilhaft sein. Insb. Datenaufteilung bietet nur HP Methoden ab einer gewissen Datenmenge Vorteile. Für kleine Mengen überwiegt der Overhead durch das Lesen mehrerer kleiner Dateien. Klassische Methoden werden durch eine Datenaufteilung nur negativ beeinflusst.
- › **Variablenauswahl beim Einlesen ist immer vorteilhaft.** High-Performance Methoden können aber bei einer **Datenaufteilung** darauf verzichten.
- › (tba: mit obigem zusammenführen) Unter den klassischen Methoden ist {data.table} mit Variablenauswahl am effizientesten und schneller als eine schlechte Anwendung von {arrow}. D.h. ein {data.table} Profi, ist besser als ein schlechter Arrow Programmierer, der aber größeres Potenzial für Verbesserungen hat. Anfängern ist eher {arrow} zu empfehlen, da in {data.table} die Variablenauswahl abhängig von dem verfügbaren Arbeitsspeicher manuell gesteuert werden muss und daher nicht so flexibel ist.
 - › 5x langsamer als {arrow}

- › Variablenauswahl ab einer gewissen Datenmenge erforderlich, da ohne 3x größer als die Eingangsdaten, dauert 2.2 min anstatt 23 Sec. -> schwierige Anwendung (Entscheidung relevanter Variablen muss vorab getroffen werden unter Beachtung der Arbeitsspeicher Restriktionen)
- › Datenaufteilung verschlechtert DT im Gegensatz zu arrow
- › {arrow} verzeiht Fehler leicht(er als DT). Denn schlecht programmiert, ist es immernoch ganz gut). Alleine durch die Datenaufteilung (vorgegeben durch die Datenbereitstellung, vorbereitet durch das FDZ), erreichen wir die Fallzahl-Berechnung in 11.73s, nur 5s über dem absoluten Optimum.
- › R-base oder readr sind immer weit abgeschlagen und zu vermeiden.
- › Wenn der RAM extrem begrenzt ist, können {poars} oder {duckdb} dennoch eingesetzt werden.